

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 5**



CONNECT TO THE INTERNET

Oleh:

Noor Khalisa

NIM. 2410817220012

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 5

Laporan Praktikum Pemrograman Mobile Modul 5: Connect to the Internet ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Noor Khalisa
NIM : 2410817220012

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Erza Raihan
NIM. 2310817210027

Irham Maulani Abdul Gani, S.Kom.,
M.Kom.
NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN	2
DAFTAR ISI.....	3
DAFTAR GAMBAR	4
DAFTAR TABEL	5
SOAL 1	6
B. Source Code	6
C. Output Program.....	29
D. Pembahasan.....	35
TAUTAN GIT.....	50

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Jawaban Soal 1 Compose	29
Gambar 2. Screenshot Hasil Jawaban Soal 1 Compose	30
Gambar 3. Screenshot Hasil Jawaban Soal 1 Compose	31
Gambar 4. Screenshot Hasil Jawaban Soal 1 Compose	32
Gambar 5. Screenshot Hasil Jawaban Soal 1 Compose	33
Gambar 6. Screenshot Hasil Jawaban Soal 1 Compose	34
Gambar 7. Screenshot Hasil Jawaban Soal 1 Compose	35

DAFTAR TABEL

Tabel 1. Source Code Movie.kt (Compose).....	6
Tabel 2. Source Code TmdbApiService.kt (Compose).....	7
Tabel 3. Source Code ApiConfig.kt (Compose)	8
Tabel 4. Source Code MovieDao.kt (Compose)	9
Tabel 5. Source Code AppDatabase.kt (Compose).....	9
Tabel 6. Source Code UserReference.kt (Compose)	10
Tabel 7. Source Code MovieRepository.kt (Compose)	11
Tabel 8. Source Code MovieViewModel.kt (Compose).....	12
Tabel 9. Source Code AppNavigation.kt (Compose).....	15
Tabel 10. Source Code HomeScreen.kt (Compose).....	16
Tabel 11. Source Code DetailScreen.kt (Compose).....	22
Tabel 12. Source Code LanguageScreen.kt (Compose).....	25
Tabel 13. Source Code MovieApplication.kt (Compose).....	27
Tabel 14. Source Code MainActivity.kt (Compose).....	27

SOAL 1

Soal Praktikum:

Lanjutkan aplikasi Android yang sudah dibuat pada Modul 4 dengan menambahkan modifikasi sesuai ketentuan berikut:

- a. Gunakan networking library seperti Retrofit atau Ktor agar aplikasi dapat mengambil data dari remote API. Dalam penggunaan networking library, sertakan generic response untuk status dan error handling pada API dan Flow untuk data stream.
- b. Gunakan KotlinX Serialization sebagai library JSON.
- c. Gunakan library seperti Coil atau Glide untuk image loading.
- d. API yang digunakan pada modul ini adalah The Movie Database (TMDB) API yang menampilkan data film. Berikut link dokumentasi API:
- e. Implementasikan konsep data persistence (aplikasi menyimpan data walau pengguna keluar dari aplikasi) dengan SharedPreferences untuk menyimpan data ringan (seperti pengaturan aplikasi) dan Room untuk data relasional.
- f. Gunakan caching strategy pada Room. Dibebaskan untuk memilih caching strategy yang sesuai, dan sertakan penjelasan kenapa menggunakan caching strategy tersebut.
- g. Untuk Modul 5, bebas memilih UI yang ingin digunakan, antara berbasis XML atau Jetpack Compose.

B. Source Code

Tabel 1. Source Code Movie.kt (Compose)

1	package com.example.modul5compose.data.model
2	
3	import androidx.room.Entity
4	import androidx.room.PrimaryKey
5	import kotlinx.serialization.SerialName
6	import kotlinx.serialization.Serializable
7	
8	@Serializable
9	data class MovieResponse (

```

10     @SerializedName("results")
11     val results: List<Movie>
12 )
13
14 @Serializable
15 @Entity(tableName = "movies")
16 data class Movie(
17     @PrimaryKey
18     @SerializedName("id")
19     val id: Int,
20
21     @SerializedName("title")
22     val title: String,
23
24     @SerializedName("overview")
25     val overview: String,
26
27     @SerializedName("poster_path")
28     val posterPath: String?,
29
30     @SerializedName("release_date")
31     val releaseDate: String?,
32
33     @SerializedName("vote_average")
34     val voteAverage: Double?
35 ) {
36
37 }

```

Tabel 2. Source Code TmdbApiService.kt (Compose)

```

1 package com.example.modul5compose.data.network
2
3 import
4     com.example.modul5compose.data.model.MovieResponse
5     import retrofit2.http.GET
6     import retrofit2.http.Query
7
8 interface TmdbApiService {
9
10     @GET("movie/popular")
11     suspend fun getPopularMovies(

```

11	@Query("api_key") apiKey: String,
12	@Query("language") language: String = "en-
13	@Query("page") page: Int = 1,
14	): MovieResponse
15	}

Tabel 3. Source Code ApiConfig.kt (Compose)

1	package com.example.modul5compose.data.network
2	
3	import kotlinx.serialization.json.Json
4	import okhttp3.MediaType.Companion.toMediaType
5	import okhttp3.OkHttpClient
6	import retrofit2.Retrofit
7	import
	retrofit2.converter.kotlinx.serialization.asConverte
	rFactory
8	
9	object ApiConfig {
10	private const val BASE_URL =
	"https://api.themoviedb.org/3/"
11	const val IMAGE_BASE_URL =
	"https://image.tmdb.org/t/p/w500"
12	
13	fun getApiService() : TmdbApiService {
14	val json = Json { ignoreUnknownKeys = true}
15	val contentType =
	"application/json".toMediaType()
16	
17	val client = OkHttpClient.Builder().build()
18	
19	val retrofit = Retrofit.Builder()
20	.baseUrl(BASE_URL)
21	.client(client)
22	
	.addConverterFactory(json.asConverterFactory(content
	Type))
23	.build()
24	

25	return
26	retrofit.create(TmdbApiService::class.java)
27	}

Tabel 4. Source Code MovieDao.kt (Compose)

1	package com.example.modul5compose.data.local
2	
3	import androidx.room.Dao
4	import androidx.room.Insert
5	import androidx.room.Query
6	import androidx.room.OnConflictStrategy
7	import com.example.modul5compose.data.model.Movie
8	import kotlinx.coroutines.flow.Flow
9	
10	@Dao
11	@JvmSuppressWildcards
12	interface MovieDao {
13	@Query("SELECT * FROM movies")
14	fun getAllMovies(): Flow<List<Movie>>
15	
16	@Insert(onConflict = OnConflictStrategy.REPLACE)
17	suspend fun insertMovies(movies: List<Movie>):
18	List<Long>
19	@Query("DELETE FROM movies")
20	suspend fun deleteAllMovies(): Int
21	}

Tabel 5. Source Code AppDatabase.kt (Compose)

1	package com.example.modul5compose.data.local
2	
3	import android.content.Context
4	import androidx.room.Database
5	import androidx.room.Room
6	import androidx.room.RoomDatabase
7	import com.example.modul5compose.data.model.Movie
8	
9	@Database(entities = [Movie::class], version = 1,
	exportSchema = false)
10	abstract class AppDatabase : RoomDatabase() {

```

11     abstract fun movieDao(): MovieDao
12
13     companion object {
14         @Volatile
15         private var INSTANCE: AppDatabase? = null
16
17         fun      getDatabase(context:      Context):
AppDatabase {
18             return INSTANCE ?: synchronized(this) {
19                 val instance = Room.databaseBuilder(
20                     context.applicationContext,
21                     AppDatabase::class.java,
22                     "movie_database"
23                 ).build()
24                 INSTANCE = instance
25                 instance
26             }
27         }
28     }
29 }

```

Tabel 6. Source Code UserReference.kt (Compose)

```

1 package com.example.modul5compose.data.local
2
3 import android.content.Context
4 import android.content.SharedPreferences
5
6 class UserPreference(context: Context) {
7     private val preference: SharedPreferences =
context.getSharedPreferences("app_settings",
Context.MODE_PRIVATE)
8
9     fun saveLanguage(languageCode: String) {
10
preference.edit().putString("SELECTED_LANGUAGE",
languageCode).apply()
11     }
12
13     fun getLanguage(): String {

```

14	return
	preference.getString("SELECTED_LANGUAGE", "en") ?:
	"en"
15	}
16	}

Tabel 7. Source Code MovieRepository.kt (Compose)

1	package com.example.modul5compose.data.repository
2	
3	import com.example.modul5compose.BuildConfig
4	import com.example.modul5compose.data.local.MovieDao
5	import com.example.modul5compose.data.model.Movie
6	import
	com.example.modul5compose.data.network.ApiConfig
7	import kotlinx.coroutines.flow.Flow
8	import kotlinx.coroutines.flow.flow
9	import timber.log.Timber
10	
11	sealed class UiState<out T : Any?> {
12	object Loading : UiState<Nothing>()
13	data class Success<out T : Any>(val data: T) :
	UiState<T>()
14	data class Error(val errorMessage: String) :
	UiState<Nothing>()
15	}
16	
17	class MovieRepository(private val movieDao: MovieDao)
	{
18	private val apiService =
	ApiConfig.getApiService()
19	
20	private val apiKey = BuildConfig.TMDB_API_KEY
21	
22	fun getPopularMovies(language: String) :
	Flow<UiState<List<Movie>>> = flow {
23	emit(UiState.Loading)
24	
25	try {
26	val response =
	apiService.getPopularMovies(apiKey = apiKey, language
	= language)

```

27         val moviesFromApi = response.results
28
29         movieDao.deleteAllMovies()
30         movieDao.insertMovies(moviesFromApi)
31         Timber.d("Data berhasil diambil dari API
dan disimpan ke database lokal.")
32     } catch (e: Exception) {
33         Timber.e("Gagal mengambil data dari API,
mencoba menggunakan data lokal. Error: ${e.message}")
34     }
35
36     try {
37         movieDao.getAllMovies().collect {
localMovies ->
38             if (localMovies.isNotEmpty()) {
39
40                 emit(UiState.Success(localMovies))
41             } else {
42                 emit(UiState.Error("Tidak ada
koneksi internet dan belum ada data yang tersimpan."))
43             }
44         } catch (e: Exception) {
45             emit(UiState.Error("Gagal membaca
database: ${e.message}"))
46         }
47     }
48 }

```

Tabel 8. Source Code MovieViewModel.kt (Compose)

```

1 package com.example.modul5compose
2
3 import androidx.lifecycle.ViewModel
4 import androidx.lifecycle.ViewModelProvider
5 import androidx.lifecycle.viewModelScope
6 import androidx.lifecycle.viewmodel.compose.viewModel
7 import com.example.modul5compose.data.model.Movie
8 import
com.example.modul5compose.data.repository.MovieRepository

```

```

9  import
   com.example.modul5compose.data.repository.UiState
10 import kotlinx.coroutines.flow.MutableStateFlow
11 import kotlinx.coroutines.flow.StateFlow
12 import kotlinx.coroutines.flow.asStateFlow
13 import kotlinx.coroutines.launch
14 import timber.log.Timber
15 import java.util.Locale
16
17 class MovieViewModel(private val repository:
   MovieRepository) : ViewModel() {
18     private val _movieState =
   MutableStateFlow<UiState<List<Movie>>>(UiState.Loading)
19     val movieState: StateFlow<UiState<List<Movie>>> =
   _movieState.asStateFlow()
20
21     private val _navigationEvent =
   MutableStateFlow<Movie?>(null)
22     val navigationEvent: StateFlow<Movie?> =
   _navigationEvent.asStateFlow()
23
24     init {
25         reloadMoviesByLocale()
26     }
27
28     fun reloadMoviesByLocale() {
29         val currentLanguage =
   Locale.getDefault().language
30         val tmdbLanguage = if (currentLanguage == "id"
   || currentLanguage == "in") "id-ID" else "en-US"
31
32         loadMovies(tmdbLanguage)
33     }
34
35     private fun loadMovies(language: String) {
36         Timber.d("Memuat data film (Bahasa:
   $language)...")
37         _movieState.value = UiState.Loading
38
39         viewModelScope.launch {

```

```

40 repository.getPopularMovies(language).collect { state
->
41         _movieState.value = state
42
43         when (state) {
44             is UiState.Success ->
45                 Timber.d("Berhasil memuat ${state.data.size} item
film.")
46             is UiState.Error ->
47                 Timber.e("Gagal memuat film: ${state.errorMessage}")
48             is UiState.Loading ->
49                 Timber.d("Status masih loading...")
50         }
51     }
52
53     fun onDetailClicked(movie: Movie) {
54         Timber.d("Tombol Detail ditekan. Navigasi ke
ID Film: ${movie.id} Judul: ${movie.title}.")
55         _navigationEvent.value = movie
56     }
57
58     fun onNavigationHandled() {
59         _navigationEvent.value = null
60     }
61
62     class MovieViewModelFactory(private val repository:
MovieRepository) : ViewModelProvider.Factory {
63         override fun <T : ViewModel> create(modelClass:
Class<T>): T {
64             if
65                 (modelClass.isAssignableFrom(MovieViewModel::class.j
ava)) {
66                 @Suppress("UNCHECKED_CAST")
67                 return MovieViewModel(repository) as T
68             }
69             throw IllegalArgumentException("Unknown
ViewModel class")

```

70 }

Tabel 9. Source Code AppNavigation.kt (Compose)

```
1 package com.example.modul5compose.ui.navigation
2
3 import androidx.compose.runtime.Composable
4 import androidx.compose.runtime.remember
5 import androidx.compose.ui.platform.LocalContext
6 import androidx.lifecycle.viewmodel.compose.viewModel
7 import androidx.navigation.NavType
8 import androidx.navigation.compose.NavHost
9 import androidx.navigation.compose.composable
10 import
    androidx.navigation.compose.rememberNavController
11 import androidx.navigation.navArgument
12 import com.example.modul5compose.MovieViewModel
13 import
    com.example.modul5compose.MovieViewModelFactory
14 import
    com.example.modul5compose.data.local.AppDatabase
15 import
    com.example.modul5compose.data.repository.MovieRepository
16 import
    com.example.modul5compose.ui.screen.DetailScreen
17 import com.example.modul5compose.ui.screen.HomeScreen
18 import
    com.example.modul5compose.ui.screen.LanguageScreen
19
20 @Composable
21 fun AppNavigation() {
22     val navController = rememberNavController()
23     val context = LocalContext.current
24
25     val database = remember {
26         AppDatabase.getDatabase(context) }
27     val repository = remember {
28         MovieRepository(database.movieDao()) }
29
30     val movieViewModel: MovieViewModel = viewModel(
31         factory = MovieViewModelFactory(repository)
```

```

30     )
31
32     NavHost(navController = navController,
startDestination = "home") {
33         composable("home") {
34             HomeScreen(navController = navController,
viewModel = movieViewModel)
35         }
36         composable(
37             route = "detail/{movieId}",
38             arguments = listOf(navArgument("movieId")
{
39                 type = NavType.IntType
40             })
41         ) { backStackEntry ->
42             val movieId =
backStackEntry.arguments?.getInt("movieId")
43             DetailScreen(
44                 navController = navController,
45                 movieId = movieId,
46                 viewModel = movieViewModel
47             )
48         }
49
50         composable("language") {
LanguageScreen(navController) }
51     }
52 }

```

Tabel 10. Source Code HomeScreen.kt (Compose)

```

1 package com.example.modul5compose.ui.screen
2
3 import androidx.compose.foundation.background
4 import androidx.compose.foundation.layout.*
5 import androidx.compose.foundation.lazy.*
6 import
androidx.compose.foundation.shape.RoundedCornerShap
e
7 import androidx.compose.material3.*
8 import androidx.compose.runtime.*
9 import androidx.compose.ui.Alignment

```



```

10 import androidx.compose.ui.Modifier
11 import androidx.compose.ui.draw.clip
12 import androidx.compose.ui.graphics.Color
13 import androidx.compose.ui.layout.ContentScale
14 import androidx.compose.ui.res.*
15 import androidx.compose.ui.text.font.FontWeight
16 import androidx.compose.ui.text.style.TextOverflow
17 import androidx.compose.ui.unit.*
18 import androidx.navigation.NavController
19 import coil.compose.AsyncImage
20 import com.example.modul5compose.MovieViewModel
21 import com.example.modul5compose.R
22 import com.example.modul5compose.data.model.Movie
23 import
    com.example.modul5compose.data.network.ApiConfig
24 import
    com.example.modul5compose.data.repository.UiState
25 import com.example.modul5compose.ui.theme.*
26
27 @Composable
28 fun HomeScreen(
29     navController: NavController,
30     viewModel: MovieViewModel
31 ) {
32     val                movieState                by
    viewModel.movieState.collectAsState()
33     val                navEvent                by
    viewModel.navigationEvent.collectAsState()
34
35     LaunchedEffect(navEvent) {
36         navEvent?.let { movie ->
37
38             navController.navigate("detail/${movie.id}")
39             viewModel.onNavigationHandled()
40         }
41     }
42     LaunchedEffect(Unit) {
43         viewModel.reloadMoviesByLocale()
44     }
45
46     Column(

```

```

47         modifier = Modifier
48             .fillMaxSize()
49             .background(LavenderBlush)
50     ) {
51         Row(
52             modifier = Modifier
53                 .fillMaxWidth()
54                 .padding(16.dp),
55             horizontalArrangement =
Arrangement.SpaceBetween,
56             verticalAlignment =
Alignment.CenterVertically
57         ) {
58             Text(
59                 text =
stringResource(R.string.app_name),
60                 color = Color.Black,
61                 fontSize = 22.sp,
62                 fontWeight = FontWeight.Bold
63             )
64             IconButton(onClick = {
65                 navController.navigate("language")
66             }) {
67                 Icon(
68                     painter =
painterResource(android.R.drawable.ic_menu_sort_alp
habetically),
69                     contentDescription = "Change
Language",
70                     tint = Watermelon
71                 )
72             }
73         }
74
75         when (val state = movieState) {
76             is UiState.Loading -> {
77                 Box(modifier =
Modifier.fillMaxSize(), contentAlignment =
Alignment.Center) {
78                     CircularProgressIndicator(color
= Watermelon)
79                 }

```

```

80         }
81         is UiState.Error -> {
82             Box(modifier
Modifier.fillMaxSize(),          contentAlignment
Alignment.Center) {
83                 Text(text
"${stringResource(R.string.error_fetch)}
${state.errorMessage}", color = Color.Red)
84             }
85         }
86         is UiState.Success -> {
87             val movieList = state.data
88
89             LazyRow(contentPadding
PaddingValues(horizontal = 16.dp),
90                 horizontalArrangement
Arrangement.spacedBy(16.dp)) {
91                 items(movieList) { movie ->
92                     MovieItemCard(
93                         movie = movie,
94                         onClick      =      {
viewModel.onDetailClicked(movie)},
95
Modifier.fillParentMaxWidth()
96                     )
97                 }
98             }
99
100             Spacer(modifier
Modifier.height(16.dp))
101
102             LazyColumn(          modifier
Modifier.weight(1f),
103                 contentPadding
PaddingValues(start = 16.dp, end = 16.dp, bottom =
16.dp),
104                 verticalArrangement
Arrangement.spacedBy(16.dp)) {
105                 items(movieList) { movie ->
106                     MovieItemCard(
107                         movie = movie,

```

```

108         onClick = {
109             viewModel.onDetailClicked(movie)},
110         Modifier.fillMaxWidth()
111     )
112 }
113 }
114 }
115 }
116 }
117
118 @Composable
119 fun MovieItemCard(movie: Movie, onClick: () -> Unit,
120     modifier: Modifier = Modifier) {
121     Card(
122         shape = RoundedCornerShape(16.dp),
123         colors = CardDefaults.cardColors(containerColor =
124             PinkChampagne),
125         elevation = CardDefaults.cardElevation(defaultElevation = 4.dp),
126         modifier = modifier
127     ) {
128         Row (
129             modifier = Modifier
130                 .fillMaxWidth()
131                 .padding(12.dp)
132         ) {
133             AsyncImage (
134                 model =
135                 "${ApiConfig.IMAGE_BASE_URL}${movie.posterPath}",
136                 contentDescription = null,
137                 contentScale = ContentScale.Crop,
138                 modifier = Modifier
139                     .size(110.dp)
140             )
141             .clip(RoundedCornerShape(16.dp))
142
143             Spacer(modifier = Modifier.width(12.dp))

```

```

142         Column(modifier                               =
143     Modifier.fillMaxWidth()) {
144             Text(
145                 text = movie.title,
146                 color = Color.Black,
147                 fontWeight = FontWeight.Bold,
148                 fontSize = 15.sp,
149                 maxLines = 1,
150                 overflow = TextOverflow.Ellipsis
151             )
152             Spacer(modifier                               =
153     Modifier.height(4.dp))
154
155             Text(
156                 text
157                 =
158     "${stringResource(R.string.label_release)}
159     ${movie.releaseDate}",
160                 color = Color.Black,
161                 fontSize = 14.sp
162             )
163
164             Spacer(modifier                               =
165     Modifier.height(8.dp))
166
167             Row (
168                 horizontalArrangement
169                 =
170     Arrangement.End,
171                 modifier
172                 =
173     Modifier.fillMaxWidth()
174             ) {
175                 Button(
176                     onClick =onClick,
177                     colors
178                     =
179     ButtonDefaults.buttonColors(
180                         containerColor
181                         =
182     Watermelon,
183                         contentColor
184                         =
185     Color.White
186                     ),
187                     contentPadding
188                     =
189     PaddingValues(12.dp, 4.dp)
190                 ) {

```

173	Text(stringResource(R.string.btn_detail), fontSize = 12.sp)
174	}
175	}
176	}
177	}
178	}
179	}

Tabel 11. Source Code DetailScreen.kt (Compose)

1	package com.example.modul5compose.ui.screen
2	
3	import androidx.compose.foundation.background
4	import androidx.compose.foundation.layout.*
5	import
	androidx.compose.foundation.rememberScrollState
6	import
	androidx.compose.foundation.shape.RoundedCornerShape
7	import androidx.compose.foundation.verticalScroll
8	import androidx.compose.material3.*
9	import androidx.compose.runtime.Composable
10	import androidx.compose.runtime.collectAsState
11	import androidx.compose.runtime.getValue
12	import androidx.compose.ui.Alignment
13	import androidx.compose.ui.Modifier
14	import androidx.compose.ui.draw.clip
15	import androidx.compose.ui.graphics.Color
16	import androidx.compose.ui.layout.ContentScale
17	import androidx.compose.ui.res.stringResource
18	import androidx.compose.ui.text.font.FontWeight
19	import androidx.compose.ui.unit.dp
20	import androidx.compose.ui.unit.sp
21	import androidx.navigation.NavController
22	import coil.compose.AsyncImage
23	import com.example.modul5compose.MovieViewModel
24	import
	com.example.modul5compose.data.network.ApiConfig
25	import
	com.example.modul5compose.data.repository.UiState
26	import com.example.modul5compose.ui.theme.*

```

27 import com.example.modul5compose.R
28
29 @Composable
30 fun DetailScreen(
31     navController: NavController,
32     movieId: Int?,
33     viewModel: MovieViewModel
34 ) {
35
36     val movieState by
37     viewModel.movieState.collectAsState()
38     val movie = if (movieState is UiState.Success) {
39         (movieState as UiState.Success).data.find {
40             it.id == movieId }
41     } else null
42
43     if (movie != null) {
44         Column(
45             modifier = Modifier
46                 .fillMaxSize()
47                 .background(LavenderBlush)
48
49             .verticalScroll(rememberScrollState())
50                 .padding(16.dp)
51         ) {
52             Box(
53                 modifier = Modifier.fillMaxWidth(),
54                 contentAlignment = Alignment.Center
55             ) {
56                 AsyncImage(
57                     model =
58                     "${ApiConfig.IMAGE_BASE_URL}${movie.posterPath}",
59                     contentDescription = "Poster
60                     ${movie.title}",
61                     contentScale =
62                     ContentScale.Crop,
63                     modifier = Modifier
64                         .size(320.dp)
65
66                     .clip(RoundedCornerShape(16.dp))
67                 )
68             }
69         }
70     }
71 }

```

```

62
63         Spacer(modifier =
Modifier.height(16.dp))
64         Text(
65             movie.title,
66             color = Color.Black,
67             fontWeight = FontWeight.Bold,
68             fontSize = 28.sp
69         )
70         Text(
71
"${stringResource(R.string.label_release_date)}
${movie.releaseDate}",
72             color = Color.Black,
73             fontSize = 16.sp,
74             modifier = Modifier.padding(top =
4.dp)
75         )
76         Text(
77
"${stringResource(R.string.label_rating)}
${movie.voteAverage}/10",
78             color = Color.Black,
79             fontSize = 16.sp,
80             modifier = Modifier.padding(top =
4.dp)
81         )
82         Text(
83             text = if
(movie.overview.isNotBlank()) {
84                 movie.overview
85             } else {
86                 "Deskripsi belum tersedia dalam
bahasa ini."
87             },
88             color = Color.Black,
89             fontSize = 15.sp,
90             lineHeight = 22.sp,
91             modifier = Modifier.padding(top =
24.dp)
92         )
93

```


94	Spacer(modifier	=
	Modifier.height(32.dp))	
95	Button(	
96	onClick	= {
	navController.popBackStack()),	
97	colors	=
	ButtonDefaults.buttonColors(containerColor	=
	Watermelon, contentColor = Color.White),	
98	modifier = Modifier.fillMaxWidth()	
99	)	{
	Text(stringResource(R.string.btn_back)) }	
100	}	
101	} else {	
102	Box(modifier = Modifier.fillMaxSize(),	
	contentAlignment = Alignment.Center) {	
103	Text((stringResource(R.string.error_not_found)),	
	color = Color.Black)	
104	}	
105	}	
106	}	

Tabel 12. Source Code LanguageScreen.kt (Compose)

1	package com.example.modul5compose.ui.screen
2	
3	import androidx.appcompat.app.AppCompatActivityDelegate
4	import androidx.compose.foundation.background
5	import androidx.compose.foundation.layout.*
6	import androidx.compose.material3.*
7	import androidx.compose.runtime.Composable
8	import androidx.compose.runtime.remember
9	import androidx.compose.ui.Alignment
10	import androidx.compose.ui.Modifier
11	import androidx.compose.ui.graphics.Color
12	import androidx.compose.ui.platform.LocalContext
13	import androidx.compose.ui.res.stringResource
14	import androidx.compose.ui.text.font.FontWeight
15	import androidx.compose.ui.unit.dp
16	import androidx.compose.ui.unit.sp
17	import androidx.core.os.LocaleListCompat
18	import androidx.navigation.NavController

```

19 import com.example.modul5compose.ui.theme.*
20 import com.example.modul5compose.R
21 import
    com.example.modul5compose.data.local.UserPreference
22
23 @Composable
24 fun LanguageScreen(navController: NavController) {
25     val context = LocalContext.current
26     val userPreference = remember {
        UserPreference(context) }
27
28     Column(
29         modifier = Modifier
30             .fillMaxSize()
31             .background(LavenderBlush)
32             .padding(24.dp),
33         verticalArrangement = Arrangement.Center,
34         horizontalAlignment = Alignment.CenterHorizontally
35     ) {
36         Text(
37             text = stringResource(R.string.setting_language),
38             color = Color.Black,
39             fontSize = 24.sp,
40             fontWeight = FontWeight.Bold,
41             modifier = Modifier.padding(bottom = 24.dp)
42         )
43         Button(
44             onClick = {
45                 userPreference.saveLanguage("en")
46                 setAppLocale("en") },
47             colors = ButtonDefaults.buttonColors(containerColor = Watermelon,
48                 contentColor = Color.White),
49             modifier = Modifier.fillMaxWidth()
50         ) {
51             Text(stringResource(R.string.btn_english)) }
52         Spacer(modifier = Modifier.height(16.dp))
53         Button(

```

53	onClick = {	
54	userPreference.saveLanguage("id")	
55	setAppLocale("id") },	
56	colors	=
	ButtonDefaults.buttonColors(containerColor	=
	Watermelon, contentColor = Color.White),	
57	modifier = Modifier.fillMaxWidth()	
58	)	{
	Text(stringResource(R.string.btn_indonesia)) }	
59	}	
60	}	
61		
62	fun setAppLocale(languageTag: String) {	
63	val localeList	=
	LocaleListCompat.forLanguageTags(languageTag)	
64	AppCompatActivity.setApplicationLocales(localeList)	
65	}	

Tabel 13. Source Code MovieApplication.kt (Compose)

1	package com.example.modul5compose
2	
3	import android.app.Application
4	import timber.log.Timber
5	
6	class MovieApplication : Application() {
7	override fun onCreate() {
8	super.onCreate()
9	Timber.plant(Timber.DebugTree())
10	}
11	}

Tabel 14. Source Code MainActivity.kt (Compose)

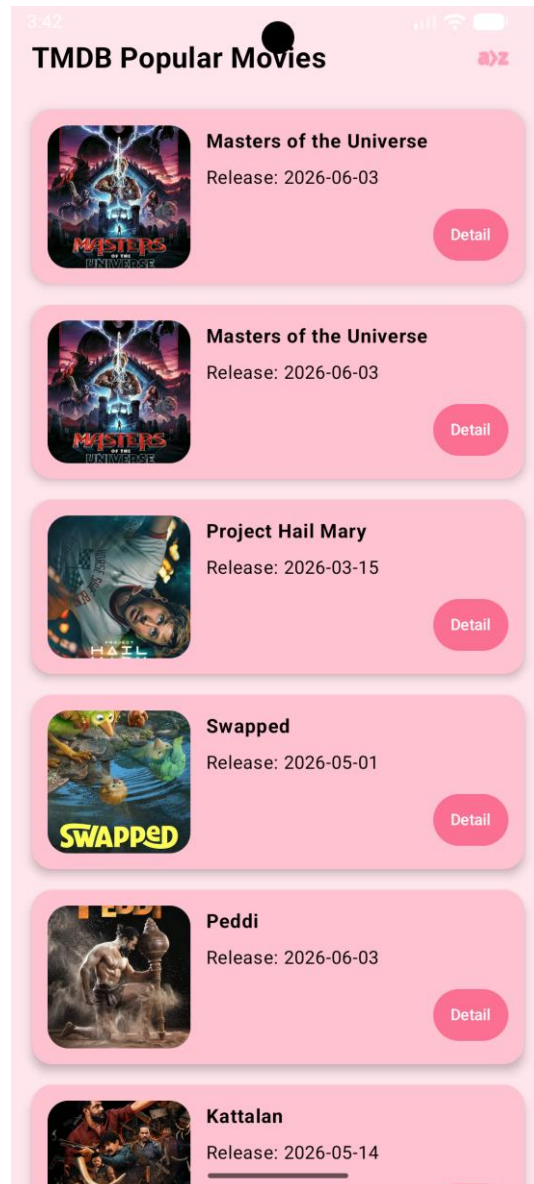
1	package com.example.modul5compose
2	
3	import android.os.Bundle
4	import androidx.activity.compose.setContent
5	import androidx.appcompat.app.AppCompatActivity
6	import androidx.appcompat.app.AppCompatActivity
7	import androidx.compose.material3.*
8	import androidx.core.os.LocaleListCompat

```

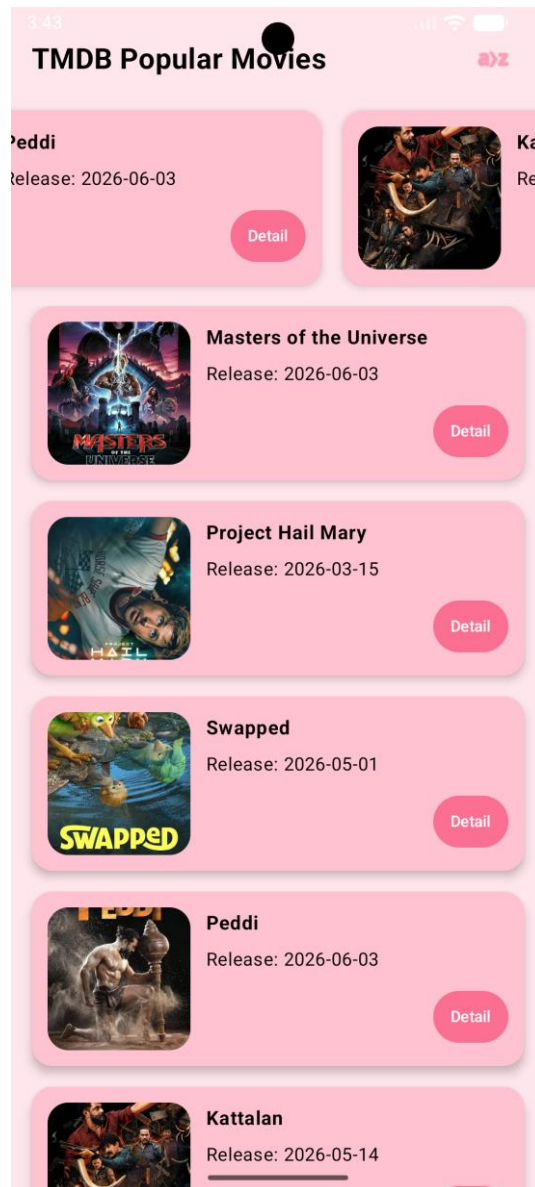
9  import
   com.example.modul5compose.data.local.UserPreference
10 import
   com.example.modul5compose.ui.navigation.AppNavigation
   on
11
12
13 class MainActivity : AppCompatActivity() {
14     override fun onCreate(savedInstanceState:
Bundle?) {
15         super.onCreate(savedInstanceState)
16
17         val userPreference = UserPreference(this)
18         val savedLanguage =
userPreference.getLanguage()
19         val localeList =
LocaleListCompat.forLanguageTags(savedLanguage)
20
AppCompatActivity.setApplicationLocales(localeList)
21
22         setContent {
23             MaterialTheme {
24                 AppNavigation()
25             }
26         }
27     }
28 }

```

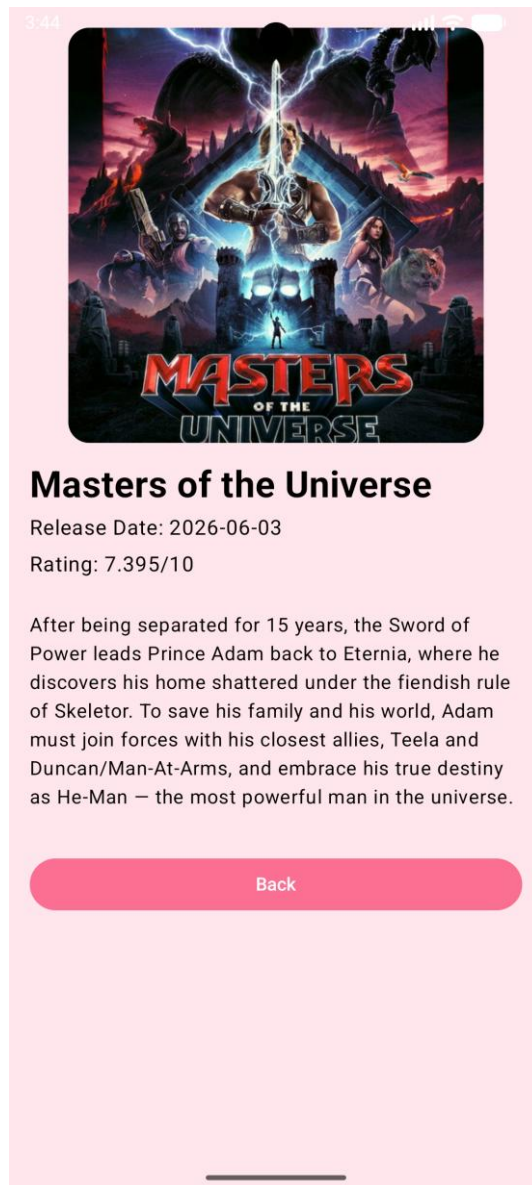
C. Output Program



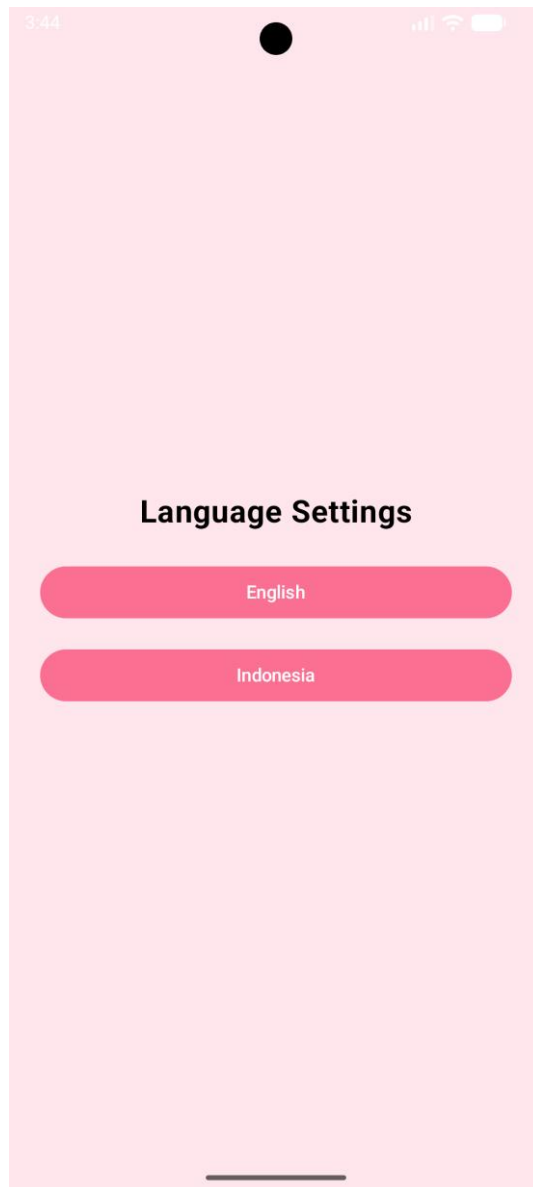
Gambar 1. Screenshot Hasil Jawaban Soal 1 Compose



Gambar 2. Screenshot Hasil Jawaban Soal 1 Compose



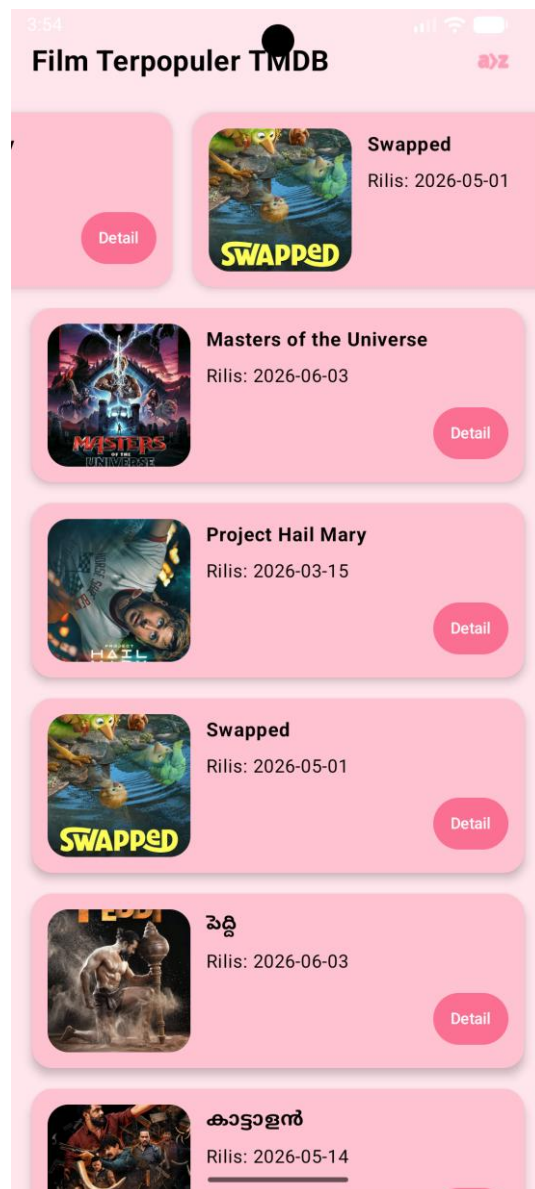
Gambar 3. Screenshot Hasil Jawaban Soal 1 Compose



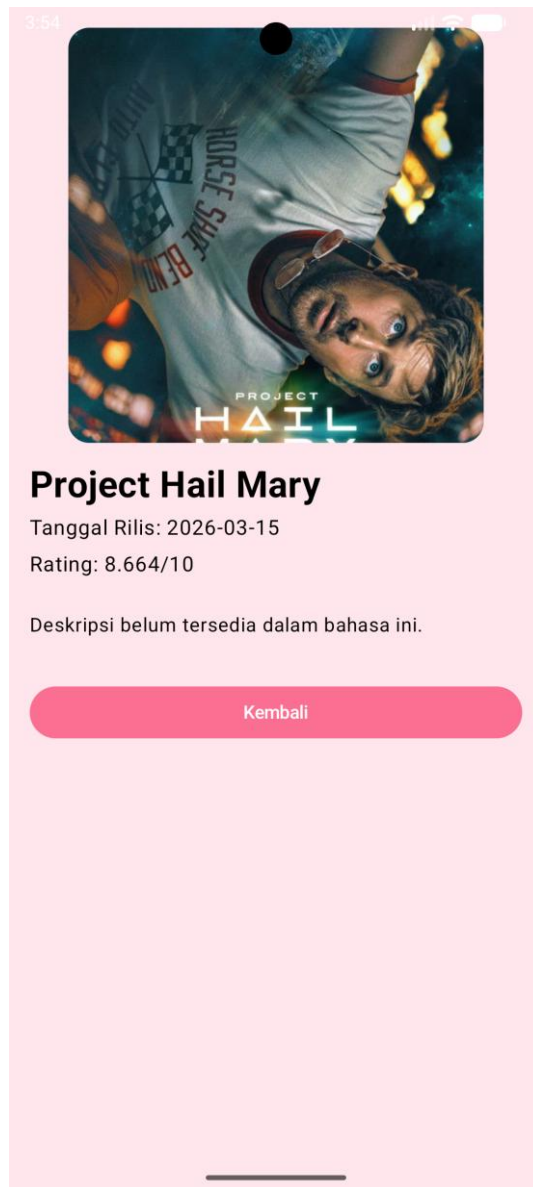
Gambar 4. Screenshot Hasil Jawaban Soal 1 Compose



Gambar 5. Screenshot Hasil Jawaban Soal 1 Compose



Gambar 6. Screenshot Hasil Jawaban Soal 1 Compose



Gambar 7. Screenshot Hasil Jawaban Soal 1 Compose

D. Pembahasan

Movie.kt

Pada baris [1] sampai [6], deklarasi *package* menunjukkan lokasi direktori file, dengan pemanggilan library bawaan *androidx.room* untuk operasi database lokal dan *kotlinx.serialization* untuk proses *parsing* data JSON dari API.

Pada baris [8] sampai [12], dideklarasikan data class `MovieResponse` dengan anotasi `@Serializable`. Class ini berfungsi sebagai wadah untuk menampung *top-level-response* dari API TMDb. Anotasi `@SerializedName("results")` memetakan *array* JSON bernama "results" agar masuk ke dalam variabel `results` yang bertipe `List<Movie>`.

Pada baris [14] sampai [36], dideklarasikan data class `Movie` yang menerapkan anotasi ganda, yaitu `@Serializable` dan `@Entity(tableName = "movies")`. Modifikasi ini membuktikan bahwa class ini dirancang dengan sangat efisien sebagai model ganda, ia bertindak sebagai model untuk mem-*parsing* objek JSON dari *remote* API, sekaligus bertindak sebagai definisi skema tabel untuk disimpan secara relasional ke dalam database lokal (Room).

Pada baris [16] dan [17], anotasi `@PrimaryKey` mendefinisikan variabel `id` sebagai kunci utama (identitas unik) bagi baris data di dalam database lokal.

Pada baris [18] sampai [33], setiap property diberikan anotasi `@SerializedName(...)`. Pendekatan ini merupakan *best practice* karena menyelaraskan format *snake_case* bawaan dari JSON API menjadi standar penamaan variabel *camelCase* di Kotlin (`posterPath`). Beberapa variabel seperti `posterPath`, `releaseDate`, dan `voteAverage` diberikan tipe data *nullable* (?) sebagai langkah pengamanan (*fail-safe*) untuk mencegah *crash* apabila *field* tersebut kebetulan kosong atau tidak dikirim oleh server API.

TmdbApiService.kt

Pada baris [1] sampai [5], deklarasi *package* diikuti dengan pemanggilan pustaka `retrofit2.http.*` serta model `MovieResponse` yang telah dibuat sebelumnya.

Pada baris [7] sampai [16], dideklarasikan sebuah interface bernama `TmdbApiService`. Kelas antarmuka ini murni bertindak sebagai kontrak spesifikasi endpoint REST API yang akan dikelola oleh library Retrofit.

Pada baris [9], anotasi `@GET("movie/popular")` menginstruksikan Retrofit untuk melakukan HTTP GET request ke endpoint relatif "movie/popular".

Pada baris [10], fungsi `getPopularMovies(...)` dideklarasikan menggunakan kata kunci `suspend`. Penggunaan `suspend` mengintegrasikan fungsi ini dengan ekosistem Kotlin Coroutines, yang memastikan bahwa proses pemanggilan jaringan (network call) berjalan secara asinkron di *background thread* dan tidak memblokir *Main Thread* (UI).

Pada baris [11] sampai [13], anotasi `@Query` disematkan pada setiap parameter argumen. Anotasi ini memerintahkan Retrofit untuk secara otomatis menempelkan argumen ini sebagai URL parameter di belakang tanda tanya (?), misalnya menjadi `.../movie/popular?api_key=XXX&language=en-US&page=1`. Nilai bawaan (*default value*) `language = "en-US"` dan `page = 1` diberikan agar pemanggilan fungsi ini menjadi fleksibel dan ringkas. Fungsi ini kemudian diinstruksikan untuk mengembalikan *generic response* `MovieResponse`.

ApiConfig.kt

Pada baris [1] sampai [7], deklarasi *package* diikuti dengan pemanggilan pustaka `okhttp3` untuk klien HTTP, `retrofit2` untuk pembangunan klien API, dan `kotlinx.serialization` sebagai pengonversi format JSON.

Pada baris [9], object `ApiConfig` dideklarasikan. Berbeda dengan class biasa, kata kunci object menerapkan design pattern Singleton. Ini menjamin bahwa di seluruh *lifecycle* aplikasi, sistem hanya akan menciptakan dan menggunakan satu instansi klien Retrofit ini, sangat menghemat memori (*resource-efficient*).

Pada baris [10] dan [11], konstanta `BASE_URL` menyimpan Alamat dasar API TMDB, sedangkan `IMAGE_BASE_URL` menyimpan tautan dasar untuk memuat aset gambar dari TMDB nanti akan digunakan oleh komponen Coil di antarmuka UI.

Pada baris [13] sampai [27], dideklarasikan fungsi `getApiService()`.

Pada baris [14], konfigurasi `Json { ignoreUnknownKeys = true }` diterapkan. Pengaturan ini mencegah aplikasi mengalami crash akibat `SerializationException` jika suatu saat antarmuka API TMDB menambahkan variabel key baru ke dalam JSON yang tidak ada terdaftar di dalam data class `Movie`.

Pada baris [17], instansi OkHttpClient dibuat sebagai mesin pemroses inti dari lalu lintas HTTP aplikasi.

Pada baris [19] sampai [23], objek Retrofit.Builder() dikonfigurasi secara berurutan, mengaitkan alamat BASE_URL, menempelkan klien OkHttpClient, dan menambahkan alat konversi (Converter Factory) dari KotlinX Serialization agar respons JSON mentah secara otomatis diterjemahkan menjadi objek Kotlin yang valid.

Pada baris [25], retrofit.create(TmdbApiService::class.java) mengeksekusi antarmuka yang sudah dibuat dan mengembalikan instansi *service* yang siap digunakan oleh lapisan Repository.

MovieDao.kt

Pada baris [1] sampai [8], deklarasi *package* menunjukkan lokasi direktori file, diikuti dengan pemanggilan library androidx.room.* untuk anotasinya dan kotlinx.coroutines.flow.Flow untuk pemrosesan aliran data.

Pada baris [10] sampai [12], dideklarasikan sebuah interface MovieDao dengan anotasi @Dao (Data Access Object). Class ini berfungsi sebagai jembatan yang berisi sekumpulan fungsi (query) untuk berinteraksi langsung dengan tabel movies di database SQLite. Anotasi @JvmSuppressWildcards disematkan secara khusus untuk mencegah *compiler* Kotlin menambahkan karakter *wildcard* (? super) saat diubah ke bahasa Java, yang sering kali memicu bentrok tipe data (Name Clash) saat Room mencoba memproses *return type* dari fungsi *suspend*.

Pada baris [13] dan [14], anotasi @Query digunakan untuk mendefinisikan perintah SQL mentah, yaitu mengambil semua data dari tabel movies. Fungsi getAllMovies() mengembalikan tipe Flow<List<Movie>>. Penggunaan Flow penting di arsitektur modern karena menciptakan data stream yang reaktif. Artinya, setiap kali ada perubahan isi data di dalam tabel database, komponen UI (Jetpack Compose) akan otomatis menerima data terbaru dan *re-render* ulang tampilannya tanpa perlu dipanggil secara manual.

Pada baris [16] dan [17], anotasi `@Insert` digunakan untuk memasukkan data film dari API ke dalam tabel. Parameter `onConflict = OnConflictStrategy.REPLACE` adalah kunci utama dari *caching strategy* aplikasi ini, jika ada film dengan ID yang sama persis masuk lagi, data lama akan ditimpa dengan data baru. Fungsi `insertMovies` menggunakan `suspend` agar berjalan di *background thread*, dan mengembalikan `List<Long>` sebagai tanda konfirmasi baris data yang berhasil disimpan.

Pada baris [19] dan [20], `@Query("DELETE FROM movies")` mendefinisikan operasi penghapusan seluruh data dari tabel. Fungsi ini juga memakai `suspend` dan mengembalikan nilai `Int` yang merepresentasikan jumlah baris yang berhasil dihapus.

AppDatabase.kt

Pada baris [1] sampai [7], deklarasi *package* diikuti pemanggilan pustaka dasar Room dan `android.content.Context`.

Pada baris [9] dan [10], class `AppDatabase` dideklarasikan sebagai abstract class yang mewarisi `RoomDatabase()`. Anotasi `@Database` mendefinisikan skema database ini, di mana `entities = [Movie::class]` memberi tahu Room bahwa tabel yang digunakan bersumber dari struktur data model `Movie`. Parameter `version = 1` merupakan versi skema saat ini, dan `exportSchema = false` menonaktifkan fitur ekspor riwayat skema karena tidak diperlukan untuk praktikum ini.

Pada baris [11], dideklarasikan fungsi abstrak `movieDao()` yang bertugas mengekspos interface `MovieDao` agar bisa diakses oleh lapisan Repository.

Pada baris [13] sampai [26], diimplementasikan *design pattern Singleton* di dalam companion object.

Pada baris [14], anotasi `@Volatile` pada variabel `INSTANCE` memastikan bahwa nilai dari variabel ini akan selalu diperbarui secara instan di main memory, sehingga perubahannya langsung terlihat oleh semua thread lain yang sedang berjalan.

Pada baris [17] sampai [25], fungsi `getDatabase(...)` memuat logika penciptaan database. Blok `synchronized(this)` menjamin bahwa proses pembuatan *database* ini

bersifat *thread-safe* untuk mencegah situasi di mana dua *thread* mencoba membuat *database* secara bersamaan. Di dalamnya, `Room.databaseBuilder(...)` bertugas membangun instansi database fisik bernama `movie_database`. Objek tunggal ini kemudian dikembalikan dan siap digunakan di seluruh aplikasi.

UserPreference.kt

Pada baris [1] sampai [5], deklarasi *package* diikuti pemanggilan library bawaan Android Context dan SharedPreferences.

Pada baris [7], class `UserPreference` dideklarasikan dengan menerima injeksi parameter context.

Pada baris [8], variabel preference menginisialisasi berkas SharedPreferences dengan nama “app_settings”. Parameter `Context.MODE_PRIVATE` sangat penting untuk keamanan data, karena membatasi hak akses file pengaturan ini agar murni hanya bisa dibaca dan ditulis oleh aplikasi ini saja, tidak bisa diintip oleh aplikasi lain.

Pada baris [10] sampai [12], fungsi `saveLanguage(...)` dideklarasikan untuk menyimpan preferensi bahasa. Perintah `preference.edit().putString(“SELECTED_LANGUAGE”, languageCode)` menempatkan nilai kode bahasa (seperti “en” atau “id”) ke dalam penyimpanan memori dengan kunci kunci “SELECTED_LANGUAGE”. Penggunaan fungsi `.apply()` di akhir memastikan bahwa proses penyimpanan (*commit*) dilakukan secara asinkron di *background*, sehingga tidak akan membuat aplikasi *lag* atau macet.

Pada baris [14] sampai [16], fungsi `getLanguage()` bertugas mengambil data bahasa yang tersimpan. Jika ternyata kuncinya tidak ditemukan (misalnya aplikasi baru pertama kali diinstal), fungsi `getString` akan menggunakan operator Elvis (`?:`) untuk mengembalikan nilai “en” sebagai pilihan bahasa *default*.

MovieRepository.kt

Pada baris [1] sampai [9], deklarasi *package* menunjukkan lokasi direktori file, diikuti dengan pemanggilan berbagai *library* penting, termasuk komponen lokal (`MovieDao`), *network* (`ApiConfig`), dan aliran data asinkron (`kotlinx.coroutines.flow.Flow`).

Pada baris [11] sampai [15], dideklarasikan sealed class UiState. Kelas ini menyediakan *generic response* untuk status dan error handling. UiState membungkus data ke dalam tiga kemungkinan status: Loading (proses berjalan), Success (berhasil membawa data), dan Error (gagal membawa pesan *error*).

Pada baris [17] sampai [21], class MovieRepository dideklarasikan dengan menerima parameter movieDao. Kelas ini bertindak sebagai *Single Source of Truth* yang menengahi konflik antara data dari API dan data dari database lokal.

Pada baris [23] dan [24], fungsi getPopularMovies(...) didesain untuk mengembalikan tipe Flow<UiState<List<Movie>>>. Eksekusi fungsi langsung diawali dengan memancarkan emit(UiState.Loading) untuk memberi sinyal kepada UI agar menampilkan indikator loading.

Pada baris [26] sampai [34], blok try-catch pertama dieksekusi untuk mencoba mengambil data dari API TMDB. Jika pengambilan berhasil (apiService.getPopularMovies), aplikasi menerapkan *Caching Strategy* jenis Refresh-on-Launch. Mekanismenya aplikasi menghapus seluruh data lama di database (movieDao.deleteAllMovies()) dan menggantinya dengan data terbaru dari API (movieDao.insertMovies(...)). Strategi ini memastikan pengguna selalu melihat daftar film paling aktual setiap kali aplikasi dimuat ulang, tanpa memakan penyimpanan lokal secara berlebih.

Pada baris [36] sampai [45], blok try-catch kedua dieksekusi untuk membaca data dari database lokal (movieDao.getAllMovies()). Ini adalah mekanisme *Offline-First* (fallback). Jika data local ada (terlepas dari apakah API tadi berhasil atau gagal karena tidak ada internet), data akan dipancarkan ke UI sebagai UiState.Success. Jika ternyata database local kosong dan API juga gagal, aplikasi memancarkan UiState.Error yang menyatakan tidak ada koneksi internet dan belum ada data tersimpan.

MovieViewModel.kt

Pada baris [1] sampai [13], deklarasi *package* diikuti dengan pemanggilan library arsitektur MVVM, Kotlin Coroutines, serta `java.util.Locale` untuk deteksi bahasa perangkat.

Pada baris [15] sampai [21], dideklarasikan `MovieViewModel` yang mewarisi arsitektur dasar `ViewModel`. Di dalamnya diterapkan konsep State Encapsulation (pola *backing property*). Status aliran data UI (`_movieState`) dan status navigasi (`_navigationEvent`) dideklarasikan sebagai `MutableStateFlow` yang bersifat `private` agar hanya bisa dimodifikasi dari dalam `ViewModel`. Keduanya diekspos sebagai `StateFlow read-only` agar bisa di-*observe* oleh UI Jetpack Compose.

Pada baris [23] sampai [25], blok inisialisasi `init` akan langsung memanggil metode `reloadMoviesByLocale()` sesaat setelah instansi `ViewModel` diciptakan di memori.

Pada baris [27] sampai [32], `fungsireloadMoviesByLocale()` mendeteksi bahasa *default* perangkat saat ini (`Locale.getDefault().language`). Jika perangkat berbahasa Indonesia (“id” atau “in”), aplikasi akan meminta data film berbahasa Indonesia (“id-ID”) dari TMDB, jika tidak, ia akan meminta versi Inggris (“en-US”).

Pada baris [34] sampai [48], fungsi `loadMovies(...)` bertugas memanggil repositori di dalam lingkup `viewModelScope.launch` agar berjalan asinkron. Fungsi ini mengumpulkan (*collect*) emisi data dari Flow repositori dan memperbarui nilai `_movieState.value` secara *real-time*. Proses ini juga dikawal oleh `Timber.d` dan `Timber.e` untuk pencatatan *log*.

Pada baris [50] sampai [56], metode interaksi dan `onNavigationHandled()` didaftarkan untuk mengontrol perpindahan layar. Sesuai prinsip *clean architecture*, UI hanya mengirimkan *event* klik ke fungsi ini, lalu `ViewModel`-lah yang memutuskan data mana yang akan dikirimkan ke layar berikutnya.

Pada baris [59] sampai [67], `MovieViewModelFactory` dibuat karena `MovieViewModel` membutuhkan parameter `rrepository` di dalam konstruktornya. *Factory* ini bertugas menciptakan dan menyuntikkan dependensi tersebut secara manual saat aplikasi berjalan.

AppNavigation.kt

Pada baris [1] sampai [14], deklarasi *package* diikuti pemanggilan library dasar Jetpack Compose, komponen sistem navigasi antarmuka (NavHost, composable, rememberNavController), serta seluruh komponen layar (HomeScreen, DetailScreen, LanguageScreen).

Pada baris [16] sampai [17], dideklarasikan fungsi *composable* @Composable fun AppNavigation().

Pada baris [20] sampai [26], dilakukan pengaturan *Dependency Injection* (DI) secara manual. Karena tidak menggunakan *library* tambahan seperti Hilt/Dagger, instansi komponen diurutkan pembuatannya di sini. Pertama mengambil AppDatabase melalui *context*, kemudian menciptakan MovieRepository dengan menyuntikkan movieDao, lalu terakhir merakit movieViewModel menggunakan movieViewModelFactory. Modifikasi remember {...} memastikan bahwa database dan repositori hanya dibuat satu kali dan tidak ikut ter-reset saat terjadi recomposition di UI.

Pada baris [28] sampai [47], NavHost mengontrol peta rute perpindahan antar layar dengan rute awalan startDestination = “home”.

Pada baris [29] sampai [31], rute statis “home” mengarah ke HomeScreen, dengan menyuntikkan pengontrol navigasi dan instansi ViewModel.

Pada baris [32] sampai [43], diimplementasikan pola pengiriman parameter (*dynamic routing*) pada rute “detail/{movieId}”. Argumen movieId ditangkap dan dikonversi secara keta menjadi bilangan bulat (). Setelah berhasil diekstrak dari backStackEntry, ID tersebut diteruskan ke DetailScreen agar layar detail bisa mencar dan menampilkan informasi film yang tepat.

Pada baris [45], rute statis “language” didaftarkan untuk menampilkan halaman pengaturan translasi bahasa tanpa memerlukan parameter tambahan.

HomeScreen.kt

Pada baris [1] sampai [17], deklarasi *package* diikuti dengan sekumpulan *import* untuk komponen Jetpack Compose. Salah satu *library* paling penting yang di-*import* pada baris ini adalah `coil.compose.AsyncImage`, menggunakan *library* Coil untuk memuat gambar dari internet secara asinkron.

Pada baris [19] sampai [22], dideklarasikan `@Composable` fun `HomeScreen` yang menerima parameter `navController` dan `viewModel`. Antarmuka ini dirancang secara reaktif untuk membaca data dari `ViewModel`.

Pada baris [23] dan [24], layar observe aliran data menggunakan `collectAsState()`. Fungsi ini membuat UI Compose akan selalu *re-render* ulang bagian layar yang relevan setiap kali ada perubahan data pada `movieState` atau `navEvent`.

Pada baris [26] sampai [31], blok `LaunchedEffect(navEvent)` ditugaskan untuk memantau aksi klik. Jika pengguna menekan sebuah film, navigasi `navController.navigate(...)` akan dipicu ke halaman detail, lalu status *event* akan segera dikembalikan ke posisi *null* menggunakan `onNavigationHandled()` untuk mencegah layar berpindah berkali-kali secara tidak sengaja.

Pada baris [33] sampai [35], blok `LaunchedEffect(Unit)` dipanggil supaya aplikasi berjalan dinamis. Setiap kali `HomeScreen` pertama kali dimuat ke layar, ia akan mengeksekusi `viewModel.reloadMoviesByLocale()` untuk memastikan data API yang diambil selalu selaras dengan pengaturan bahasa (*locale*) terbaru di perangkat.

Pada baris [63] sampai [106], diimplementasikan arsitektur *State Management* antarmuka menggunakan blok pengecekan `when` (`val state = movieState`). Ini merupakan bentuk penerapan langsung dari *generic response* `UiState`.

Pada baris [64] sampai [68], jika statusnya `UiState.Loading`, aplikasi akan menampilkan ikon putaran `CircularProgressIndicator` di tengah layar untuk memberi tahu pengguna bahwa aplikasi sedang mengunduh data dari internet.

Pada baris [69] sampai [73], jika statusnya `UiState.Error`, antarmuka akan mencetak pesan kesalahan dengan warna teks merah agar mudah disadari oleh pengguna.

Pada baris [74] sampai [104], jika statusnya `UiState.Success`, data film diekstrak ke dalam variabel `movieList`. Data ini kemudian dirender ke dalam komponen daftar `LazyRow` (untuk gulir horizontal di bagian atas) dan `LazyColumn` (untuk gulir vertikal di bagian bawah). Setiap iterasi akan memanggil komponen `MovieItemCard`.

Pada baris [109] sampai [180], fungsi `@Composable fun MovieItemCard(...)` dideklarasikan sebagai komponen kartu yang dapat didaur ulang (*reusable*). Kartu didesain menggunakan warna latar `PinkChampagne` dengan sudut melengkung.

Pada baris [121] sampai [129], komponen inti `AsyncImage` dari `Coil` digunakan untuk mengunduh dan menampilkan poster film. Property model diisi dengan penggabungan tautan dasar gambar () dan Alamat spesifik sampul film dari API (). `Coil` akan secara otomatis menangani proses download, *caching* gambar di memori, dan memasangnya ke dalam kotak ukuran 110dp dengan sangat efisien.

Pada baris [133] sampai [140], elemen `Text` untuk judul ditambahkan parameter pengaman `maxLines = 1` dan `overflow = TextOverflow.Ellipsis`. Ini memastikan jika nama film terlalu panjang, teksnya tidak akan merusak tatanan kartu dan secara otomatis dipotong menggunakan tanda titik tiga (...).

Pada baris [154] sampai [163], ditambahkan `Button` dengan label teks spesifik dari `stringResource(R.string.btn_detail)` untuk mengakomodasi berbagai bahasa. Saat tombol ini ditekan, akan dijalankan fungsi `onClick` yang telah disuntikkan dari parameter di atasnya.

DetailScreen.kt

Pada baris [1] sampai [17], *import* library `Jetpack Compose` dan navigasi mencakup komponen visual untuk membangun tata letak yang responsif serta `coil.compose.AsyncImage` untuk menampilkan poster film dengan performa tinggi.

Pada baris [19] sampai [23], fungsi `@Composable fun DetailScreen` dideklarasikan dengan parameter `navController`, `movieId` (yang dikirim dari rute navigasi), dan

viewModel. Fungsi ini bertanggung jawab untuk merender halaman detail film secara dinamis berdasarkan ID yang dipilih.

Pada baris [25] sampai [28], layar melakukan observasi terhadap viewModel.movieState menggunakan collectAsState(). Logika pemfilteran data dilakukan dengan perintah (movieState as UiState.Success).data.find { it.id == movieId }. Ini adalah teknik efisien untuk mendapatkan satu objek film spesifik dari daftar yang sudah tersimpan di dalam *state* ViewModel, tanpa perlu memanggil API ulang atau kueri database tambahan.

Pada baris [30] sampai [85], blok if (movie != null) memastikan bahwa konten visual hanya di-*render* jika data film yang diminta berhasil ditemukan. Jika data belum tersedia, aplikasi akan mengarahkan pengguna ke blok else untuk menampilkan pesan kesalahan yang informatif menggunakan stringResource(R.string.error_not_found).

Pada baris [33] sampai [36], tata letak utama dibungkus dengan Column dan dimodifikasi menggunakan verticalScroll(rememberScrollState()). Modifikasi ini vital karena *overview* sering kali memiliki panjang teks yang bervariasi, dengan *scroll* aktif, konten yang panjang tidak akan terpotong oleh batas bawah layar perangkat.

Pada baris [37] sampai [48], komponen AsyncImage digunakan kembali untuk menampilkan poster film. Berbeda dengan di HomeScreen, poster di sini ditampilkan dengan ukuran yang lebih besar (320.dp) untuk menonjolkan detail visual film.

Pada baris [51] sampai [81], teks informasi film seperti judul, tanggal rilis, rating, dan deskripsi disusun secara vertical. Penggunaan stringResource(R.string.label_...) memastikan seluruh label teks di halaman ini mendukung pelokalan (multi-bahasa) yang konsisten dengan pengaturan di LanguageScreen. Khusus untuk deskripsi film, ditambahkan logika kondisional, jika deskripsi kosong, aplikasi akan menampilkan teks *fallback* agar pengguna tidak melihat ruang kosong yang membingungkan.

Pada baris [83] sampai [87], tombol “Back” ditempatkan di paling bawah halaman. Fungsi onClick = { navController.popBackStack() } merupakan mekanisme navigasi standar yang membuang tumpukan halaman detail dari *back stack*, sehingga aplikasi

secara otomatis mengembalikan pengguna ke halaman HomeScreen tanpa harus memuat ulang seluruh aplikasi.

LanguageScreen.kt

Pada baris [1] sampai [14], deklarasi *package* diikuti dengan pemanggilan *library* penting, terutama `androidx.appcompat.app.AppCompatActivity` dan `androidx.core.os.LocaleListCompat` yang merupakan komponen kunci untuk mengubah bahasa aplikasi secara *runtime* tanpa perlu *restart* aplikasi.

Pada baris [17] dan [18], fungsi `@Composable` fun `LanguageScreen` diinisialisasi dengan menggunakan `UserPreference`. Objek ini bertindak sebagai *wrapper* untuk `SharedPreferences`, yang memungkinkan aplikasi menyimpan pilihan bahasa pengguna (“en” atau “id”) ke dalam memori permanen perangkat secara lokal.

Pada baris [20] sampai [27], tata letak halaman diatur menggunakan `Column` dengan parameter `Arrangement.Center` dan `Alignment.CenterHorizontally`. Pengaturan ini memaksa seluruh elemen antarmuka, mulai dari judul hingga tombol, untuk selalu terkunci simetris di poros tengah layar, memberikan estetika yang bersih untuk sebuah panel pengaturan.

Pada baris [34] sampai [51], terdapat dua komponen `Button` untuk pilihan Bahasa Inggris dan Indonesia. Implementasi *event handling* pada atribut `onClick` melakukan dua aksi sekaligus. Pertama, menyimpan kode bahasa ke `SharedPreferences` agar preferensi pengguna tetap tersimpan meskipun aplikasi ditutup. Kedua, memanggil fungsi `setAppLocale(...)` untuk memperbarui bahasa secara langsung.

Pada baris [55] sampai [58], fungsi `setAppLocale(languageTag: String)` dideklarasikan sebagai fungsi utilitas prosedural. Perintah `LocaleListCompat.forLanguageTags(languageTag)` menerima string kode bahasa dan mengonversinya menjadi objek *locale* yang valid. Selanjutnya, `AppCompatActivity.setApplicationLocales(localeList)` dieksekusi untuk memaksa sistem Android melakukan *refresh* pada seluruh sumber daya string di aplikasi.

MovieApplication.kt

Pada baris [1] sampai [5], deklarasi *package* diikuti dengan pemanggilan *library* standar android.app.Application dan library *logging* timber.log.Timber.

Pada baris [7] sampai [11], dideklarasikan class MovieApplication yang mewarisi class dasar Application. Dalam arsitektur Android, kelas ini adalah komponen paling pertama yang dieksekusi oleh sistem sesaat sebelum aktivitas atau layar apa pun ditampilkan. Ini tempat yang paling direkomendasikan untuk melakukan inisialisasi global.

Pada baris [8] sampai [10], metode onCreate() di-*override*. Di dalamnya, perintah Timber.plant(Timber.DebugTree()) dieksekusi. Perintah ini menanam fungsi *logging* ke seluruh siklus hidup aplikasi. Dengan cara ini, tidak perlu lagi menulis Log.d secara manual di setiap class, setiap pesan *log* yang dikirim melalui Timber akan tercatat dengan rapi di *Logcat* Android Studio lengkap dengan nama kelas dan baris kodenya, yang sangat membantu proses *debugging* aplikasi.

MainActivity.kt

Pada baris [1] sampai [8], deklarasi *package* diikuti dengan pemanggilan *library* penting untuk mendukung arsitektur Jetpack Compose, navigasi, dan fitur perubahan bahasa dinamis (AppCompatDelegate).

Pada baris [11], class MainActivity dideklarasikan dengan mewarisi AppCompatActivity. Pewarisan ini bersifat mutlak karena aplikasi menggunakan fitur *dynamic locale switching* melalui AppCompatDelegate, yang kompatibilitas penuhnya hanya didukung oleh class ini.

Pada baris [12] sampai [24], metode onCreate(savedInstanceState: Bundle?) di-*override*. Sebelum merender antarmuka, aplikasi melakukan pemulihan preferensi bahasa. UserPreference(this) memanggil pengaturan yang pernah disimpan oleh pengguna, getLanguage() mengambil kode bahasa terakhir. AppCompatDelegate.setApplicationLocales(localeList) kemudian menerapkan bahasa

tersebut ke konteks aplikasi. Langkah ini memastikan bahwa saat aplikasi terbuka, langsung menggunakan bahasa yang dipilih pengguna pada sesi sebelumnya, tanpa perlu pengaturan ulang manual.

Pada baris [20] sampai [24], perintah `setContent { ... }` dipanggil sebagai *root* dari seluruh antarmuka aplikasi. Di dalam blok inilah fungsi `AppNavigation()` dipanggil, yang akan merakit dan menampilkan seluruh rute layar (Home, Detail, Language) ke dalam kanvas visual perangkat menggunakan teknologi Jetpack Compose.

TAUTAN GIT

<https://github.com/lisaryuna/Praktikum-Mobile>